ESE 5460 HW4

Jinhao You
you7@seas.upenn.com

December 2025

Solution 1

(i) Create the dataset. We construct a dataset of 10,000 MNIST images by selecting exactly 1000 images for each digit class. Each MNIST image is first downsampled from 28×28 to 14×14, and then binarized by the rule

$$x_{jk} = \begin{cases} 1, & \text{if } x_{jk} > 128, \\ 0, & \text{otherwise.} \end{cases}$$

The following Python code performs the required resizing, binarization and balanced subsampling:

Listing 1: Building the 10k binarized MNIST dataset

```python
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import torch

def build_mnist_10k_dataset(root="./data"):
    # 1. Downsample 28x28 -> 14x14, then binarize
    binarize_thresh = 128.0 / 255.0
    transform = transforms.Compose([
        transforms.Resize((14, 14)),
        transforms.ToTensor(),
        transforms.Lambda(lambda x: (x >
            binarize_thresh).float())
    ])

    # 2. Load MNIST training set
    full_train = datasets.MNIST(
        root=root,
        train=True,
        download=True,
        transform=transform
    )

```

```python
    # 3. Select 1000 samples per class
    indices_per_class = {k: [] for k in range(10)}
    for idx, (_, label) in enumerate(full_train):
        label = int(label)
        if len(indices_per_class[label]) < 1000:
            indices_per_class[label].append(idx)
        if all(len(v) == 1000 for v in indices_per_class.values()):
            break

    # Merge indices and create subset
    selected_indices = []
    for k in range(10):
        selected_indices.extend(indices_per_class[k])

    print(f"Total selected samples: {len(selected_indices)}")
    mnist_10k = Subset(full_train, selected_indices)
    return mnist_10k


def build_mnist_val_dataset(root="./data"):
    transform = transforms.Compose([
        transforms.Resize((14, 14)),
        transforms.ToTensor(),
    ])
    mnist_val = datasets.MNIST(root=root, train=False, download=True,
                               transform=transform)
    return mnist_val
```

(ii) Encoder, decoder, and training. The encoder consists of two fully–connected layers. The first layer maps the $14 \times 14 = 196$-dimensional input to 128 units with tanh nonlinearity, and the second layer maps 128 units to 16 outputs (no nonlinearity). These 16 outputs are split into an 8-dimensional mean vector $\mu_u(x)$ and an 8-dimensional log-variance vector $\log \sigma_u(x)^2$, giving the diagonal Gaussian posterior

$$p_u(z \mid x) = \mathcal{N}\big(\mu_u(x), \sigma_u(x)^2 I\big), \qquad z \in R^8.$$

The decoder takes $z \in R^8$, maps it to 128 hidden units with tanh nonlinearity, and then to 196 output units with sigmoid nonlinearity, producing Bernoulli probabilities for each binary MNIST pixel.

We optimize the evidence lower bound (ELBO)

$$\ell(u, v; x) = E_{z \sim p_u(z|x)}[\log p_v(x \mid z)] - \mathrm{KL}(p_u(z \mid x) \,\|\, \mathcal{N}(0, I)),$$

and the empirical objective

$$\frac{1}{n}\sum_{i=1}^{n}\ell(u,v;x^i).$$

The KL divergence between diagonal Gaussians is

$$\mathrm{KL}(p_u(z \mid x)\,\|\,\mathcal{N}(0,I)) = -\frac{1}{2}\sum_{i=1}^{8}\left[1+\log\sigma_u(x)_i^2-\mu_u(x)_i^2-\sigma_u(x)_i^2\right].$$

Since MNIST is binarized, we use the Bernoulli decoder likelihood

$$\log p_v(x \mid z) = \sum_{i=1}^{196}\left[x_i\log y_i + (1-x_i)\log(1-y_i)\right].$$

We sample using the reparameterization trick

$$z = \mu_u(x) + \sigma_u(x)\odot\epsilon, \qquad \epsilon\sim\mathcal{N}(0,I),$$

and follow the problem instruction to use $S = 2$ Monte-Carlo samples of $z \sim p_u(z \mid x)$ for the expectation term.

The full PyTorch implementation is shown below.

Listing 2: VAE implementation for part (ii).

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

from dataloader import build_mnist_10k_dataset


# ========================
#  Encoder & Decoder
# ========================

class Encoder(nn.Module):
    def __init__(self, input_dim=196, hidden_dim=128, latent_dim=8):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        # Output = [mu, logvar]
        self.fc2 = nn.Linear(hidden_dim, 2 * latent_dim)
        self.tanh = nn.Tanh()

    def forward(self, x):
        """
```

```python
        x: (batch_size, 196)
        Returns:
          mu: (batch_size, 8)
          logvar: (batch_size, 8)
        """
        h = self.tanh(self.fc1(x))
        h2 = self.fc2(h)
        mu, logvar = torch.chunk(h2, 2, dim=1)
        return mu, logvar


class Decoder(nn.Module):
    def __init__(self, latent_dim=8, hidden_dim=128, output_dim=196):
        super().__init__()
        self.fc1 = nn.Linear(latent_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)
        self.tanh = nn.Tanh()
        self.sigmoid = nn.Sigmoid()

    def forward(self, z):
        """
        z: (batch_size, 8)
        Returns y in (0,1)^{196}
        """
        h = self.tanh(self.fc1(z))
        y = self.sigmoid(self.fc2(h))
        return y


# ========================
#  Reparameterization & ELBO
# ========================

def reparameterize(mu, logvar, num_samples=2):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn(num_samples, mu.size(0), mu.size(1), device=mu.device)
    return mu.unsqueeze(0) + std.unsqueeze(0) * eps


def vae_loss(x, mu, logvar, decoder, num_samples=2):
    batch_size, dim = x.shape

    # 1. sample z
    z_samples = reparameterize(mu, logvar, num_samples)

    # 2. reconstruction term
    recon_log_probs = []
    x_expand = x.unsqueeze(0)
```

```python

    for s in range(num_samples):
        z = z_samples[s]
        y = decoder(z)
        eps = 1e-8
        log_pxz = x * torch.log(y + eps) + (1 - x) * torch.log(1 - y + eps)
        log_pxz = log_pxz.sum(dim=1)
        recon_log_probs.append(log_pxz)

    recon_log_probs = torch.stack(recon_log_probs, dim=0)
    recon_term = recon_log_probs.mean(dim=0)
    recon_term_mean = recon_term.mean()

    # 3. KL(q||p)
    kl = -0.5 * torch.sum(
        1 + logvar - mu.pow(2) - torch.exp(logvar), dim=1
    )
    kl_mean = kl.mean()

    # 4. ELBO
    elbo = recon_term - kl
    loss = -elbo.mean()

    return loss, recon_term_mean.item(), kl_mean.item()


# ========================
#  Training Loop
# ========================

def train_vae(train_loader, device="cpu", lr=1e-3, num_epochs=30):
    encoder = Encoder().to(device)
    decoder = Decoder().to(device)
    optimizer = optim.Adam(
        list(encoder.parameters()) + list(decoder.parameters()), lr=lr
    )

    recon_history = []
    kl_history = []

    encoder.train()
    decoder.train()

    for epoch in range(num_epochs):
        for batch_idx, (x, _) in enumerate(train_loader):
            x = x.view(x.size(0), -1).to(device).float()

```

```python
            mu, logvar = encoder(x)
            loss, recon_avg, kl_avg = vae_loss(
                x, mu, logvar, decoder, num_samples=2
            )

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            recon_history.append(recon_avg)
            kl_history.append(kl_avg)

            if (batch_idx + 1) % 50 == 0:
                print(f"Epoch {epoch+1}, Step {batch_idx+1}, "
                      f"Loss={loss.item():.4f}, Recon={recon_avg:.4f}, KL={kl_avg:.4f}")

    return encoder, decoder, recon_history, kl_history


def main():
    device = "cuda" if torch.cuda.is_available() else "cpu"

    mnist_10k = build_mnist_10k_dataset(root="./data")
    train_loader = DataLoader(mnist_10k, batch_size=128, shuffle=True)

    encoder, decoder, recon_history, kl_history =
        train_vae(train_loader, device=device)

    torch.save(encoder.state_dict(), "encoder_hw4_p1.pt")
    torch.save(decoder.state_dict(), "decoder_hw4_p1.pt")


if __name__ == "__main__":
    main()
```

(iii) Plotting the two terms of the ELBO. Recall that for a single datum x we defined

$$\ell(u, v; x) = \underbrace{E_{z\sim p_u(z|x)}[\log p_v(x \mid z)]}_{\text{reconstruction term}} - \underbrace{\mathrm{KL}(p_u(z \mid x) \,\|\, \mathcal{N}(0, I))}_{\text{KL term}}.$$

During training we record, for every weight update, the minibatch averages of the reconstruction term and the KL term. After training, we plot these two quantities as functions of the number of weight updates. The helper `save_elbo_two_plots` creates one PDF for each term:

The resulting plots are included in the report as Fig. 1 and Fig. 2:

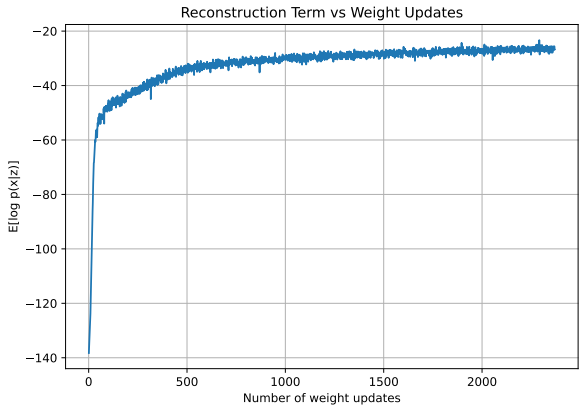

Figure 1: Reconstruction term $E_{z\sim p_u(z|x)}[\log p_v(x \mid z)]$ as a function of the number of weight updates.

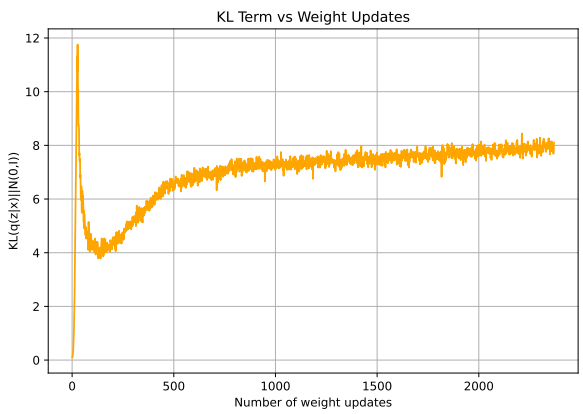

Figure 2: KL term $\mathrm{KL}(p_u(z \mid x) \,\|\, \mathcal{N}(0, I))$ as a function of the number of weight updates.

Why the KL term increases during training. Although the KL divergence in the ELBO,

$$\mathrm{KL}\big(q(z \mid x) \,\|\, \mathcal{N}(0, I)\big),$$

acts as a regularizer that encourages the posterior $q(z \mid x)$ to remain close to the standard normal prior, it is *not* optimized in isolation. The ELBO objective,

$$E_{z\sim q(z|x)}[\log p(x \mid z)] - \mathrm{KL}\big(q(z \mid x) \,\|\, \mathcal{N}(0, I)\big),$$

contains two competing forces: the reconstruction term pushes the encoder to encode as much information about x into z as possible, while the KL term encourages $q(z \mid x)$ to resemble the prior.

During the early stages of training, the decoder is still weak and cannot produce accurate reconstructions if $q(z \mid x)$ is constrained to remain close to the prior. As a result, the reconstruction term dominates the optimization, driving the encoder to increase the magnitude of $\mu(x)$ or adjust $\sigma(x)$ so that more information is transmitted through the latent representation. Both behaviors cause $q(z \mid x)$ to deviate further from the standard normal prior, which *increases* the KL divergence according to

$$\mathrm{KL} = -\frac{1}{2} \sum_i \left[1 + \log \sigma_i(x)^2 - \mu_i(x)^2 - \sigma_i(x)^2\right].$$

Thus, the KL term rising is expected: the model temporarily favors richer latent encodings to improve reconstruction quality. Only later in training, once the decoder improves, does the KL term tend to stabilize as the encoder and decoder move toward a balance between accurate reconstruction and adherence to the prior. Hence, an increasing KL term does not indicate incorrect behavior but is a natural consequence of the competition between the two components of the ELBO.

(iv) Visualizing 8 random reconstructions. We randomly select eight binarized MNIST images, encode them into the latent space, decode them back through the VAE, and display each original image next to its reconstruction. This provides a qualitative evaluation of how well the decoder has learned the generative process. The Python code below generates an 8×2 grid (original on the left, reconstruction on the right) and saves the resulting figure as `hw4_reconstruction_8.pdf`.

Listing 3: Plotting 8 random original/reconstructed MNIST images.

```python
import torch
import matplotlib.pyplot as plt
from vae_model import Encoder, Decoder
from dataloader import build_mnist_10k_dataset


def plot_8_reconstructions(encoder, decoder, dataset, device="cpu",
                           save_path="reconstruction_8.pdf"):
    """
    Randomly pick 8 images from the dataset, reconstruct them via
    encoder+decoder, and plot each original next to its reconstruction.
    """
    encoder.eval()
```

```python
    decoder.eval()

    idxs = torch.randint(0, len(dataset), (8,))

    originals = []
    recons = []

    for idx in idxs:
        x, _ = dataset[idx]                 # (1,14,14)
        x_flat = x.view(1, -1).to(device)   # (1,196)

        mu, logvar = encoder(x_flat)
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        z = mu + std * eps                  # reparameterization

        y = decoder(z)                      # (1,196)
        y_img = y.view(14, 14).detach().cpu()

        originals.append(x.view(14,14).cpu())
        recons.append(y_img)

    fig, axes = plt.subplots(8, 2, figsize=(4, 16))
    for i in range(8):
        axes[i, 0].imshow(originals[i], cmap="gray")
        axes[i, 0].set_title("Original")
        axes[i, 0].axis("off")

        axes[i, 1].imshow(recons[i], cmap="gray")
        axes[i, 1].set_title("Reconstructed")
        axes[i, 1].axis("off")

    plt.tight_layout()
    plt.savefig(save_path)
    plt.close()


def main():
    device = "cuda" if torch.cuda.is_available() else "cpu"
    encoder = Encoder().to(device)
    decoder = Decoder().to(device)

    encoder.load_state_dict(torch.load("encoder_hw4_p1.pt", map_location=device))
    decoder.load_state_dict(torch.load("decoder_hw4_p1.pt", map_location=device))

    dataset = build_mnist_10k_dataset("./data")

```

```python
    plot_8_reconstructions(
        encoder, decoder, dataset, device=device,
        save_path="hw4_reconstruction_8.pdf"
    )


if __name__ == "__main__":
    main()
```

The resulting image is shown in Fig. 5. Each row contains the original MNIST digit (left) and its reconstruction (right). The VAE successfully captures the coarse structure of each digit while smoothing out fine details, as expected for a model with a low-dimensional latent space.

(v) Sampling from the generative model. After training the VAE, we can sample new digits directly from the generative model by drawing latent vectors $z \sim \mathcal{N}(0, I_8)$ and passing them through the decoder. The following script draws eight latent samples, generates the corresponding images, and saves them in a 2×4 grid as `hw4_p1_random_samples.pdf`.

Listing 4: Sampling from the generative model for part (v).

```python
import torch
import matplotlib.pyplot as plt
from vae_model import Encoder, Decoder


def sample_from_vae(decoder, device="cpu",
                    save_path="vae_random_samples.pdf"):
    """
    Sample z ~ N(0, I) and generate MNIST-style images with
        the decoder.
    """
    decoder.eval()

    # Draw 8 latent vectors z ~ N(0, I)
    z = torch.randn(8, 8).to(device)    # (8, latent_dim)
    y = decoder(z)                      # (8, 196)
    imgs = y.view(-1, 14, 14).detach().cpu()

    fig, axes = plt.subplots(2, 4, figsize=(8, 4))
    for i, ax in enumerate(axes.flatten()):
        ax.imshow(imgs[i], cmap="gray")
        ax.set_title(f"z sample {i+1}")
        ax.axis("off")

    plt.tight_layout()
    plt.savefig(save_path)
    plt.close()

```

```python
    print(f"Saved randomly generated samples to: {save_path}")


def main():
    device = "cuda" if torch.cuda.is_available() else "cpu"

    decoder = Decoder().to(device)
    decoder.load_state_dict(
        torch.load("decoder_hw4_p1.pt", map_location=device)
    )
    decoder.eval()

    sample_from_vae(
        decoder, device=device,
        save_path="hw4_p1_random_samples.pdf"
    )


if __name__ == "__main__":
    main()
```

The synthesized samples are shown in Fig. 3. The images resemble plausible MNIST digits, demonstrating that the decoder together with the Gaussian prior defines a reasonable generative model.

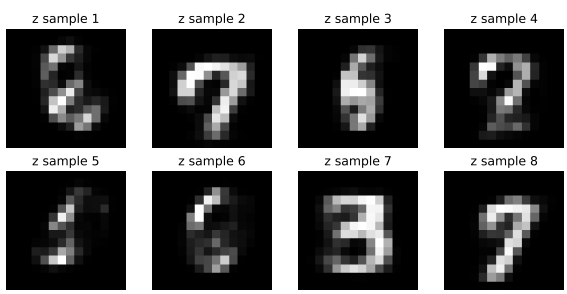

Figure 3: Eight samples drawn from the generative model: $z \sim \mathcal{N}(0, I)$ followed by the decoder.

(vi) Training vs. validation reconstruction log-likelihood. We now evaluate the validation performance of the generative model. After *every 100 weight updates* we compute the average validation reconstruction log-likelihood $E[\log p(x \mid z)]$ on a minibatch of 100 validation images. At the same time we record the reconstruction term on every training minibatch (one point per weight update). The code below reuses the VAE loss and adds validation evaluation and a joint plot of train vs. validation curves.

Listing 5: Training with validation reconstruction log-likelihood for part (vi).

```python
import torch
import torch.nn as nn
import torch.optim as optim
from vae_model import Encoder, Decoder
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
from dataloader import build_mnist_10k_dataset, build_mnist_val_dataset


def reparameterize(mu, logvar, num_samples=2):
    """
    Sample num_samples latent vectors from q(z|x):
      z = mu + sigma * eps,  eps ~ N(0,I),
    where sigma = exp(0.5 * logvar).
    Returns shape (num_samples, batch_size, latent_dim).
    """
    batch_size, latent_dim = mu.shape
    std = torch.exp(0.5 * logvar)
    eps = torch.randn(num_samples, batch_size, latent_dim, device=mu.device)
    mu_expand = mu.unsqueeze(0)
    std_expand = std.unsqueeze(0)
    z = mu_expand + std_expand * eps
    return z


def vae_loss(x, mu, logvar, decoder, num_samples=2):
    """
    Approximate:
      ELBO(x)     1/S sum_s log p(x|z^(s)) - KL(q(z|x)||N(0,I)).
    Returns:
      loss:      negative ELBO (scalar),
      recon_term: mean reconstruction term,
      kl_term:    mean KL term.
    """
    batch_size, dim = x.shape
    z_samples = reparameterize(mu, logvar, num_samples=num_samples)

    recon_log_probs = []
    x_expand = x.unsqueeze(0)

    for s in range(num_samples):
        z = z_samples[s]
        y = decoder(z)
        eps = 1e-8
```

```python
        log_pxz = x * torch.log(y + eps) + (1 - x) * torch.log(1 - y + eps)
        log_pxz = log_pxz.sum(dim=1)
        recon_log_probs.append(log_pxz)

    recon_log_probs = torch.stack(recon_log_probs, dim=0)
    recon_term = recon_log_probs.mean(dim=0)
    recon_term_mean = recon_term.mean()

    kl = -0.5 * torch.sum(
        1 + logvar - mu.pow(2) - torch.exp(logvar), dim=1
    )
    kl_mean = kl.mean()

    elbo = recon_term - kl
    loss = -elbo.mean()

    return loss, recon_term_mean.item(), kl_mean.item()


def compute_val_loglik(encoder, decoder, val_loader,
                       device="cpu", num_samples=2):
    """
    Compute the average reconstruction log-likelihood E[log p(x|z)]
    on the first batch (100 images) from the validation set.
    """
    encoder.eval()
    decoder.eval()

    for x, _ in val_loader:
        x = x.view(x.size(0), -1).to(device).float()
        mu, logvar = encoder(x)
        _, recon_avg, _ = vae_loss(
            x, mu, logvar, decoder, num_samples=num_samples
        )
        return recon_avg

    return None


def train_vae(train_loader, val_loader, device="cpu",
              lr=1e-3, num_epochs=10):
    encoder = Encoder().to(device)
    decoder = Decoder().to(device)
    params = list(encoder.parameters()) + list(decoder.parameters())
    optimizer = optim.Adam(params, lr=lr)

    recon_history = []         # train reconstruction term
```

```python
    (every update)
    kl_history = []            # train KL term (every update)
    val_loglik_history = []    # validation log-likelihood (every 100 updates)
    val_steps = []             # corresponding global update indices

    encoder.train()
    decoder.train()

    global_step = 0

    for epoch in range(num_epochs):
        for batch_idx, (x, _) in enumerate(train_loader):
            x = x.view(x.size(0), -1).to(device).float()

            mu, logvar = encoder(x)
            loss, recon_avg, kl_avg = vae_loss(
                x, mu, logvar, decoder, num_samples=2
            )

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            global_step += 1

            recon_history.append(recon_avg)
            kl_history.append(kl_avg)

            # Every 100 updates, evaluate on 100 validation images
            if global_step % 100 == 0:
                val_ll = compute_val_loglik(
                    encoder, decoder, val_loader, device=device
                )
                val_loglik_history.append(val_ll)
                val_steps.append(global_step)
                print(f"[Val] step {global_step}, "
                      f"val log p(x|z) = {val_ll:.4f}")

    return (encoder, decoder, recon_history,
            kl_history, val_steps, val_loglik_history)


def plot_train_val_recon(recon_history, val_steps, val_loglik_history,
                         save_path="train_val_recon.pdf"):
    steps = range(1, len(recon_history) + 1)
```

```python

    plt.figure(figsize=(8, 5))

    # Training reconstruction term: one point per update
    plt.plot(steps, recon_history,
             label="Train reconstruction term")

    # Validation log-likelihood: one point every 100 updates
    plt.scatter(val_steps, val_loglik_history,
                label="Validation log-likelihood",
                marker="o")

    plt.xlabel("Number of weight updates")
    plt.ylabel("E[log p(x|z)]")
    plt.title("Train reconstruction term vs validation log-likelihood")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(save_path)
    plt.close()
    print(f"Saved train/validation plot to: {save_path}")


def main():
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print("Using device:", device)

    mnist_10k = build_mnist_10k_dataset(root="./data")
    mnist_val = build_mnist_val_dataset(root="./data")

    val_loader = DataLoader(
        mnist_val,
        batch_size=100,  # exactly 100 validation images
        shuffle=False
    )

    train_loader = DataLoader(
        mnist_10k,
        batch_size=128,
        shuffle=True,
        drop_last=False
    )

    num_epochs = 30
    lr = 1e-3
    (encoder, decoder, recon_history, kl_history,
     val_steps, val_loglik_history) = train_vae(
        train_loader, val_loader, device=device,
        lr=lr, num_epochs=num_epochs
```

```
    )

    plot_train_val_recon(
        recon_history, val_steps, val_loglik_history,
        save_path="hw4_p1_train_val_recon.pdf"
    )


if __name__ == "__main__":
    main()
```

The resulting comparison between training and validation reconstruction log-likelihoods is shown in Fig. 4. The solid curve corresponds to the reconstruction term on the training minibatches (one point per weight update), while the markers show the validation log-likelihood estimated every 100 updates on 100 validation images.

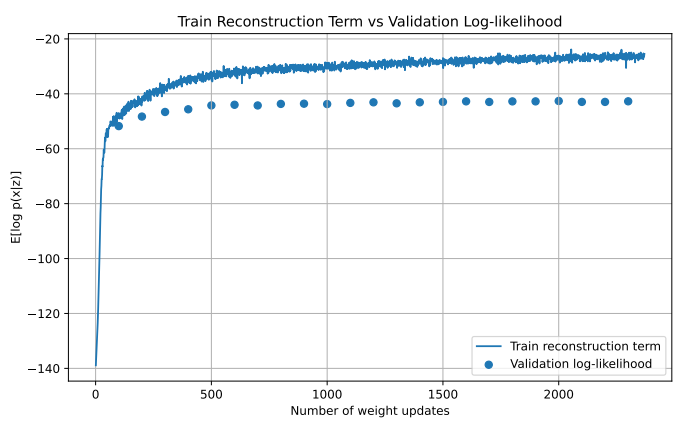

Figure 4: Reconstruction term on the training minibatches vs. validation reconstruction log-likelihood $E[\log p(x \mid z)]$ as training progresses.

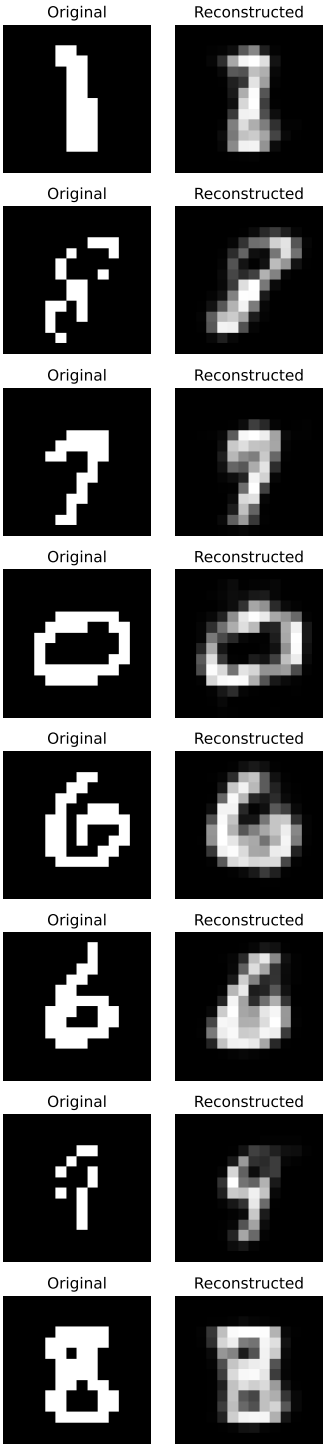

Figure 5: Eight randomly selected MNIST images (left column) and the corresponding VAE reconstructions (right column).

Appendix: Complete VAE Implementation

This appendix contains all Python code used for Parts (ii)–(vi), including the encoder/decoder architecture, ELBO computation, training loop, visualization of reconstruction quality, generative sampling, and validation log-likelihood evaluation.

A. Encoder and Decoder Architecture

```python
import torch
import torch.nn as nn

class Encoder(nn.Module):
    def __init__(self, input_dim=196, hidden_dim=128, latent_dim=8):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, 2 * latent_dim)
        self.tanh = nn.Tanh()

    def forward(self, x):
        h = self.tanh(self.fc1(x))
        h2 = self.fc2(h)
        mu, logvar = torch.chunk(h2, 2, dim=1)
        return mu, logvar


class Decoder(nn.Module):
    def __init__(self, latent_dim=8, hidden_dim=128, output_dim=196):
        super().__init__()
        self.fc1 = nn.Linear(latent_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)
        self.tanh = nn.Tanh()
        self.sigmoid = nn.Sigmoid()

    def forward(self, z):
        h = self.tanh(self.fc1(z))
        y = self.sigmoid(self.fc2(h))
        return y
```

B. Reparameterization, ELBO Loss, and Training Loop

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
```

```python
import matplotlib.pyplot as plt
from dataloader import build_mnist_10k_dataset, build_mnist_val_dataset
from vae_model import Encoder, Decoder


def reparameterize(mu, logvar, num_samples=2):
    batch_size, latent_dim = mu.shape
    std = torch.exp(0.5 * logvar)
    eps = torch.randn(num_samples, batch_size, latent_dim, device=mu.device)
    return mu.unsqueeze(0) + std.unsqueeze(0) * eps


def vae_loss(x, mu, logvar, decoder, num_samples=2):
    z_samples = reparameterize(mu, logvar, num_samples)
    x_expand = x.unsqueeze(0)

    recon_log_probs = []
    eps = 1e-8

    for s in range(num_samples):
        z = z_samples[s]
        y = decoder(z)
        log_pxz = x * torch.log(y + eps) + (1 - x) * torch.log(1 - y + eps)
        recon_log_probs.append(log_pxz.sum(dim=1))

    recon_log_probs = torch.stack(recon_log_probs, dim=0)
    recon_term = recon_log_probs.mean(dim=0)
    recon_term_mean = recon_term.mean()

    kl = -0.5 * torch.sum(
        1 + logvar - mu.pow(2) - torch.exp(logvar), dim=1
    )
    kl_mean = kl.mean()

    loss = -(recon_term - kl).mean()
    return loss, recon_term_mean.item(), kl_mean.item()


def compute_val_loglik(encoder, decoder, val_loader, device="cpu", num_samples=2):
    encoder.eval()
    decoder.eval()

    for x, _ in val_loader:
        x = x.view(x.size(0), -1).to(device).float()
        mu, logvar = encoder(x)
```

```python
        _, recon_avg, _ = vae_loss(x, mu, logvar, decoder, num_samples=num_samples)
        return recon_avg

    return None


def train_vae(train_loader, val_loader, device="cpu", lr=1e-3, num_epochs=30):
    encoder = Encoder().to(device)
    decoder = Decoder().to(device)

    optimizer = optim.Adam(
        list(encoder.parameters()) + list(decoder.parameters()),
        lr=lr
    )

    recon_history = []
    kl_history = []
    val_steps = []
    val_loglik_history = []

    encoder.train()
    decoder.train()

    global_step = 0

    for epoch in range(num_epochs):
        for batch_idx, (x, _) in enumerate(train_loader):
            x = x.view(x.size(0), -1).to(device).float()

            mu, logvar = encoder(x)
            loss, recon_avg, kl_avg = vae_loss(x, mu, logvar, decoder)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            global_step += 1
            recon_history.append(recon_avg)
            kl_history.append(kl_avg)

            if global_step % 100 == 0:
                val_ll = compute_val_loglik(encoder, decoder, val_loader, device)
                val_loglik_history.append(val_ll)
                val_steps.append(global_step)

```

```python
    return (
        encoder, decoder,
        recon_history, kl_history,
        val_steps, val_loglik_history
    )


def plot_train_val_recon(recon_history, val_steps, val_loglik_history,
                         save_path="train_val_recon.pdf"):
    steps = range(1, len(recon_history) + 1)

    plt.figure(figsize=(8,5))
    plt.plot(steps, recon_history, label="Train reconstruction term")
    plt.scatter(val_steps, val_loglik_history,
                label="Validation log-likelihood", marker="o")

    plt.xlabel("Number of weight updates")
    plt.ylabel("E[log p(x|z)]")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(save_path)
    plt.close()
```

C. Reconstruction Visualization (Part iv)

```python
import torch
import matplotlib.pyplot as plt
from vae_model import Encoder, Decoder
from dataloader import build_mnist_10k_dataset


def plot_8_reconstructions(encoder, decoder, dataset, device="cpu",
                           save_path="reconstruction_8.pdf"):
    encoder.eval()
    decoder.eval()

    idxs = torch.randint(0, len(dataset), (8,))
    originals, recons = [], []

    for idx in idxs:
        x, _ = dataset[idx]
        x_flat = x.view(1, -1).to(device).float()

```

```
        mu, logvar = encoder(x_flat)
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        z = mu + std * eps

        y = decoder(z)
        y_img = y.view(14, 14).cpu()

        originals.append(x.view(14,14).cpu())
        recons.append(y_img)

    fig, axes = plt.subplots(8, 2, figsize=(4, 16))
    for i in range(8):
        axes[i, 0].imshow(originals[i], cmap="gray")
        axes[i, 0].set_title("Original")
        axes[i, 0].axis("off")

        axes[i, 1].imshow(recons[i], cmap="gray")
        axes[i, 1].set_title("Reconstructed")
        axes[i, 1].axis("off")

    plt.tight_layout()
    plt.savefig(save_path)
    plt.close()
```

D. Generative Sampling (Part v)

```
import torch
import matplotlib.pyplot as plt
from vae_model import Decoder

def sample_from_vae(decoder, device="cpu",
                    save_path="vae_random_samples.pdf"):
    decoder.eval()

    z = torch.randn(8, 8).to(device)
    y = decoder(z)
    imgs = y.view(-1, 14, 14).cpu()

    fig, axes = plt.subplots(2, 4, figsize=(8, 4))
    for i, ax in enumerate(axes.flatten()):
        ax.imshow(imgs[i], cmap="gray")
        ax.set_title(f"z sample {i+1}")
        ax.axis("off")

    plt.tight_layout()
    plt.savefig(save_path)
    plt.close()
```

E. Validation Log-likelihood (Part vi)

```python
import torch
import torch.nn as nn
import torch.optim as optim
from vae_model import Encoder ,Decoder
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
from dataloader import build_mnist_10k_dataset
    ,build_mnist_val_dataset



def reparameterize(mu, logvar, num_samples=2):

    batch_size, latent_dim = mu.shape
    std = torch.exp(0.5 * logvar)                    #
    eps = torch.randn(num_samples, batch_size, latent_dim,
        device=mu.device)
    mu_expand = mu.unsqueeze(0)                      # (1, B, D)
    std_expand = std.unsqueeze(0)                    # (1, B, D)
    z = mu_expand + std_expand * eps                 # (S, B, D)
    return z


def vae_loss(x, mu, logvar, decoder, num_samples=2):
    batch_size, dim = x.shape
    # ---- 1.         z ----
    z_samples = reparameterize(mu, logvar,
        num_samples=num_samples)  # (S,B,8)

    # ---- 2. reconstruction log-likelihood ----
    # x: (B,D) -> (1,B,D)
    x_expand = x.unsqueeze(0)  # (1,B,D)
    recon_log_probs = []

    for s in range(num_samples):
        z = z_samples[s]                        # (B,8)
        y = decoder(z)                          # (B,196), in
            (0,1)
        # Bernoulli log p(x|z) = sum_i x_i log y_i +
            (1-x_i) log(1-y_i)
        eps = 1e-8
        log_pxz = x * torch.log(y + eps) + (1 - x) *
            torch.log(1 - y + eps)
        log_pxz = log_pxz.sum(dim=1)            # (B,)
        recon_log_probs.append(log_pxz)

    # recon_term
```

```python
    recon_log_probs = torch.stack(recon_log_probs, dim=0)  # (S,B)
    recon_term = recon_log_probs.mean(dim=0)  # (B,)
    recon_term_mean = recon_term.mean()  # scalar

    # KL = -0.5 *   _i   (1 + log  _i  ^2 -  _i  ^2 -  _i  ^2)
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - torch.exp(logvar), dim=1)  # (B,)
    kl_mean = kl.mean()

    elbo = recon_term - kl           # (B,)
    loss = -elbo.mean()              #

    return loss, recon_term_mean.item(), kl_mean.item()


def compute_val_loglik(encoder, decoder, val_loader, device="cpu", num_samples=2):
    """
                        100                          E[log p(x|z)]
    """
    encoder.eval()
    decoder.eval()

    for x, _ in val_loader:
        x = x.view(x.size(0), -1).to(device).float()  # (B,196)
        mu, logvar = encoder(x)
        _, recon_avg, _ = vae_loss(x, mu, logvar, decoder, num_samples=num_samples)
        return recon_avg

    return None


def train_vae(train_loader, val_loader, device="cpu", lr=1e-3, num_epochs=10):
    encoder = Encoder().to(device)
    decoder = Decoder().to(device)
    params = list(encoder.parameters()) + list(decoder.parameters())
    optimizer = optim.Adam(params, lr=lr)

    recon_history = []
    kl_history = []

    val_loglik_history = []
```

```python
    val_steps = []

    encoder.train()
    decoder.train()

    global_step = 0

    for epoch in range(num_epochs):
        for batch_idx, (x, _) in enumerate(train_loader):
            x = x.view(x.size(0), -1).to(device).float()

            mu, logvar = encoder(x)
            loss, recon_avg, kl_avg = vae_loss(
                x, mu, logvar, decoder, num_samples=2
            )

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            global_step += 1

            recon_history.append(recon_avg)
            kl_history.append(kl_avg)


            if global_step % 100 == 0:
                val_ll = compute_val_loglik(encoder, decoder, val_loader, device=device)
                val_loglik_history.append(val_ll)
                val_steps.append(global_step)
                print(f"[Val] step {global_step}, val log p(x|z) = {val_ll:.4f}")

            if (batch_idx + 1) % 50 == 0:
                print(f"Epoch {epoch+1}, Step {batch_idx+1}, "
                      f"GlobalStep={global_step}, Loss={loss.item():.4f}, "
                      f"Recon={recon_avg:.4f}, KL={kl_avg:.4f}")

    return encoder, decoder, recon_history, kl_history, val_steps, val_loglik_history


def plot_train_val_recon(recon_history, val_steps, val_loglik_history,
                         save_path="train_val_recon.pdf"):
    steps = range(1, len(recon_history) + 1)
```

```python

    plt.figure(figsize=(8,5))

    plt.plot(steps, recon_history, label="Train reconstruction term")

    plt.scatter(val_steps, val_loglik_history, label="Validation log-likelihood",
                marker="o")

    plt.xlabel("Number of weight updates")
    plt.ylabel("E[log p(x|z)]")
    plt.title("Train Reconstruction Term vs Validation Log-likelihood")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(save_path)
    plt.close()


def main():
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print("Using device:", device)

    mnist_10k = build_mnist_10k_dataset(root="./data")
    mnist_val = build_mnist_val_dataset(root="./data")

    val_loader = DataLoader(
        mnist_val,
        batch_size=100,
        shuffle=False
    )

    batch_size = 128
    train_loader = DataLoader(
        mnist_10k,
        batch_size=batch_size,
        shuffle=True,
        drop_last=False
    )

    num_epochs = 30
    lr = 1e-3
    encoder, decoder, recon_history, kl_history, val_steps, val_loglik_history = train_vae(
        train_loader,
        val_loader,
        device=device,
        lr=lr,
```

```
        num_epochs=num_epochs
    )

    plot_train_val_recon(recon_history, val_steps, val_loglik_history,
                         save_path="hw4_p1_train_val_recon.pdf")


if __name__ == "__main__":
    main()
```